

# Sol Language and the AI-Native Engineering Workflow

---

*Design review notes on outcome-based development, semantic verification, and reducing human code-review dependence*

## **Central thesis**

Sol is already well aligned with a future where humans review intent, constraints, evidence, and outcomes rather than routinely reading implementation diffs. Its strongest primitives are effects, capabilities, contracts, semantic identity, and structured inspection. The next major step is to make the change itself a first-class, evidence-carrying object.

**Repository reviewed:** <https://github.com/keyboardspecialist/sol-lang>

*Prepared: September 25, 2026*

# 1. Executive assessment

Sol is not merely an "AI-friendly syntax" language. Its design attacks the information problem that makes conventional code review necessary: reviewers currently reconstruct intent, authority, side effects, contracts, architectural impact, and behavioral risk from implementation text. Sol already exposes several of those facts directly to the compiler and tooling.

| Workflow goal                                  | Current Sol fit                                                                      | Assessment              |
|------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------|
| Expose intent without reading implementation   | Nominal/refined types, Result/Option, contracts, explicit effects and capabilities   | Strong                  |
| Make consequences machine-readable             | Effect rows and capability authority are explicit                                    | Strong foundation       |
| Give agents bounded semantic context           | Structured inspection and semantic IDs exist; context packets are planned            | Partial                 |
| Review semantic change instead of textual diff | Semantic delta work is planned but not yet the default artifact                      | Not yet delivered       |
| Prevent silent weakening of safeguards         | Roadmap recognizes protected tests/policy and approval of authority/contract changes | Designed, not delivered |
| Risk-based human approval                      | Facts exist that can drive policy; no full risk engine yet                           | Partial                 |
| Outcome-based production verification          | Capability boundary is a natural instrumentation point                               | Mostly missing          |
| Measure AI-maintenance improvement             | M5-style controlled experiments are planned                                          | Not yet demonstrated    |

## 2. What Sol already gets right

### 2.1 Effects and capabilities create a machine-readable blast radius

The effects/capabilities split may be the most important idea in the language for an AI-native workflow. Effects describe what a computation may do; capabilities describe what authority it has been granted to do it. That distinction lets tooling answer a question that human reviewers currently spend enormous effort inferring: "What can this change actually affect?"

```
effects      = operations this computation might perform
capabilities = authority granted to perform those operations
```

This creates the basis for automated escalation. A large implementation can become low-interest if its authority, effects, public API, and contracts are unchanged. A tiny change can become high-risk if it introduces a new payment, filesystem, network, identity, or data-mutation authority.

```

SEMANTIC CHANGE REPORT
Public API:      unchanged
Authority:      unchanged
Possible effects: unchanged
Contracts:      strengthened: 2, weakened: 0
Dependencies:    +0
Evidence:       284 / 284 tests pass

```

```

Decision: eligible for low-risk policy lane

```

## 2.2 Semantic identity is the route away from text-centric development

Stable top-level semantic identities are more important than they first appear. They allow tooling and agents to reason about a declaration as a semantic object rather than as a line range in a file. The long-term agent instruction should look more like "modify this semantic object while preserving these invariants" than "edit lines 183-217."

```

target: inventory.reserve
preserve:
  public signature
  external effects
  authority requirements
change:
  retry postcondition

```

The current distinction between persistent top-level identities and snapshot-local expression/pattern IDs is sensible. A future middle layer of stable semantic anchors could make fine-grained semantic edits more resilient without requiring globally persistent IDs for every AST node.

## 2.3 Contracts and evidence states are well suited to agent verification

Sol is strongest when it is explicit about epistemic status. "Declared", "runtime-checked", "proved under assumptions", and "unknown" are much safer categories for AI systems than prose such as "this is backward compatible." Unknown should remain a first-class result rather than something an LLM is encouraged to fill with confidence.

```

Behavior preserved:   UNKNOWN
Authority preserved:  PROVED BY STATIC ANALYSIS
API preserved:       PROVED BY SEMANTIC COMPARISON
Contract preserved:  PROVED BY COMPARISON
Performance preserved: TESTED

```

## 3. The missing layer: make changes first-class

Sol currently has program contracts. An AI-native development process also needs a contract on the change itself. The key workflow artifact should describe intent, target scope, invariants that must remain true, required evidence, and forbidden forms of weakening.

```

change reserve_retry_fix {
  intent {
    Retry of an existing reservation must not consume inventory.
  }

  target { reservation.reserve }

  preserve {
    public_api
    capability_authority
    effect_surface
    error_variants
  }

  require {
    existing_tests
    reservation_retry_suite
    interpreter_differential
  }

  forbid {
    contract_weakening
    authority_expansion
  }
}

```

This does not necessarily need to be ordinary Sol source syntax. A separately protected change-contract artifact may be better because the implementation agent should not be able to silently redefine the goal or weaken the verification standard it is judged against.

### Target review artifact

A human should be able to approve most low-risk changes from an intent/evidence summary containing: intent, semantic delta, authority delta, contract delta, test evidence, runtime evidence, and explicit unknowns. Opening the implementation should be an exception, not the default.

## 4. Recommended product priorities

### 4.1 Ship semantic diff / impact analysis early

Given two checked snapshots, produce a conservative semantic change report. This is the feature that can begin replacing git diff as the primary human review surface.

- Public declarations added, removed, or modified.
- Signature and type changes.
- Call-edge changes and newly reachable behavior.
- Effect-row changes.
- Capability requirement and authority changes.
- Contract strengthening or weakening.

- Known behavior changes versus explicitly unknown consequences.

## 4.2 Make bounded context packets a first-class agent primitive

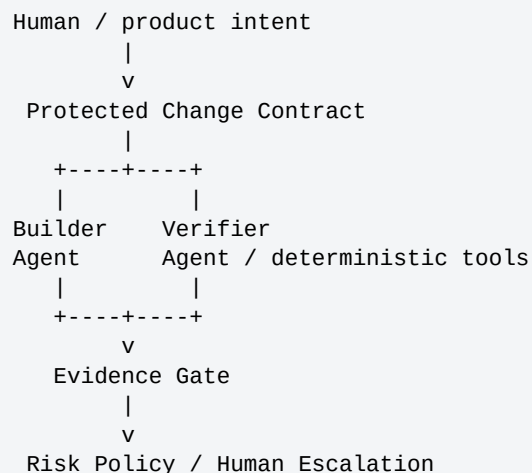
A command such as ``sol context <semantic-id>`` should return the minimum semantically relevant packet for a task: declaration, relevant types, callers/callees, effect row, capability roots, contracts, relevant tests, and omitted/unknown information. Crucially, the packet should explain why each included element was selected.

## 4.3 Produce a reproducible evidence bundle

Every accepted change should be able to emit a content-addressed attestation tying together the source snapshot, compiler version, policy version, change contract, semantic delta, tests/evaluations, and results. "Tests passed" should refer to a reproducible evidence object, not a sentence generated by the same agent that wrote the code.

## 4.4 Separate builder authority from verifier authority

The implementation agent should not be able to change tests, weaken contracts, relax architecture policy, and then certify its own change. Verification inputs need ownership and authorization separate from implementation. This is the workflow equivalent of Sol capabilities: the builder receives only the authority required to implement, while the verifier receives authority to evaluate and attest.



## 4.5 Turn architecture rules into executable policy

Function contracts are not enough. Sol should support repository- or subsystem-level architecture constraints so senior engineers can encode design judgment once rather than repeat it in every review. The capability model makes this more powerful than a conventional import linter because policy can constrain both dependency direction and authority direction.

```

architecture Checkout {
  module domain {
    may_import { core }
    may_effect { panic }
    may_capability { }
  }

  module application {
    may_import { domain }
    may_capability {
      inventory.read
      inventory.reserve
    }
  }
}

```

## 5. Use the capability boundary as observability infrastructure

Sol has an unusually valuable opportunity: important external behavior already crosses a structured host/capability boundary. That boundary can become a trusted source of runtime evidence instead of relying on application code to remember to emit the right logs and metrics.

```

effect event
  caller:          sem:1:abcd...
  operation:       payments.capture
  capability_root: checkout.payment
  outcome:        success
  duration:       34 ms
  contract_status: pass

```

A deployment system can then compare semantic effects and outcomes between the previous release and a candidate. This brings the workflow all the way from static semantics to production behavior.

```

Previous release
  capture failure: 0.14%

```

```

Candidate
  capture failure: 0.13%

```

```

Unexpected effects:      none
New authority exercised: none
Contract violations:     none

```

## 6. Do not make formal proof the prerequisite

Solver-backed verification, ghost state, and formal proof can become valuable evidence tiers, but they should not block the core workflow. Most of the process transformation can happen with strong types, explicit effects and authority, executable contracts, structured semantic IR, semantic diff, protected tests, independent execution, and production evidence.

**Practical rule**

Formal proof should strengthen the evidence ladder, not define the minimum viable AI-native workflow. The key is truthful evidence classification and explicit unknowns.

## 7. Preserve independent verification throughout the stack

Sol already demonstrates a useful compiler-engineering principle with independent evaluators/differential checks: do not make one implementation verify itself. Carry that principle upward into the software workflow.

|            |                            |
|------------|----------------------------|
| Agent A    | implementation             |
| Compiler   | static semantics           |
| Verifier A | contracts                  |
| Verifier B | generated / property tests |
| Runtime    | effect trace               |
| Deployment | outcome comparison         |

Confidence should come from independent evidence converging, not from a single model producing code, tests, review comments, and the declaration of success.

## 8. The experiment that matters most

The central empirical question for Sol is not how quickly an agent can generate code. It is whether a competent engineer can make the correct change decision without reading the implementation.

**Proposed north-star metric**

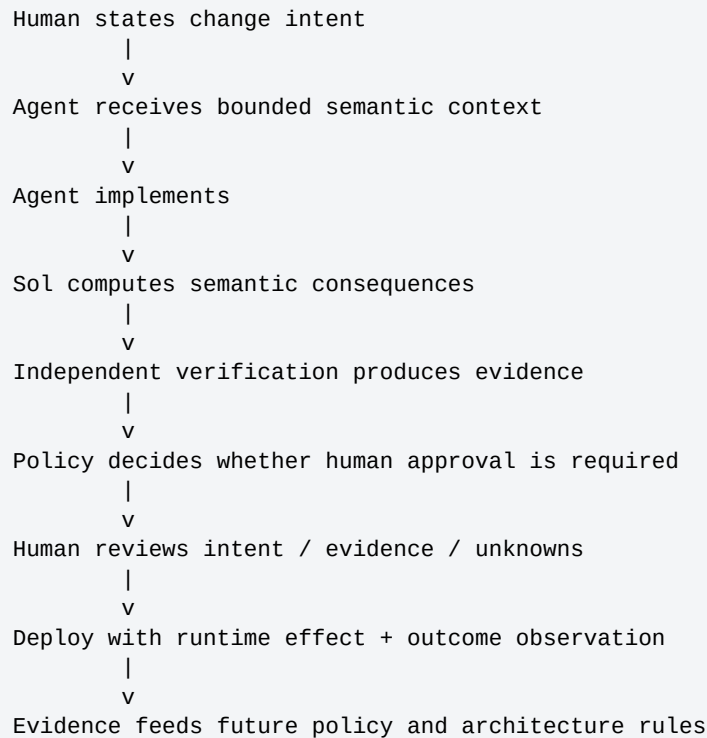
How often can a reviewer correctly approve or reject a change using Sol's intent, semantic delta, authority delta, contract delta, evidence, and unknowns - without opening the implementation?

A controlled study should compare conventional source/diff review against the Sol workflow on held-out maintenance tasks. Useful measurements include:

- Correct approval/rejection decision.
- Time to decision.
- Escaped regression rate.
- False rejection rate.
- Number of implementation views requested.
- Context tokens consumed by the agent and reviewer.
- Number of implementation/verification iterations.
- Usefulness of diagnostics and explicit unknowns.
- False-confidence rate.

The most distinctive metric may be the implementation-view rate. If Sol can reduce the percentage of changes for which a human must inspect source while preserving or improving correctness, it has demonstrated a genuinely new software-engineering model.

## 9. Target end-state workflow



At that point source code is still important, but it is no longer the organization's primary review interface. It becomes one implementation artifact inside a larger semantic and evidentiary system.

## 10. Bottom line

Sol already has several of the right primitives for outcome-based, AI-heavy software engineering: effects, capabilities, contracts, semantic identities, structured inspection, deterministic semantics, and an explicit distinction between known and unknown evidence. Those features can make the consequences of a change more reviewable than its implementation text.

The project becomes truly differentiated when the normal development unit stops being "a branch containing a diff" and becomes "a change contract with a semantic delta and reproducible evidence bundle." The most important near-term work is therefore not merely compilation or additional syntax. It is the tooling and policy layer that lets organizations safely stop reading most code.

### Reference note

These notes synthesize a review of the public Sol repository and its roadmap/design material as of September 25, 2026, together with the preceding discussion about AI-native software engineering workflows. Repository: <https://github.com/keyboardspecialist/sol-lang>